

1 FindBack — Campus Lost & Found

Student Name: [Enter your name]

Student ID: [Enter your student ID]

Course: CSCD602: Advanced Software Engineering

Project type: Individual 48-hour software engineering capstone

1.1 1. Project Title, Problem Statement, Aim and Objectives

FindBack is a web application that helps a university community report, discover, and safely reclaim lost property. Informal posts in group chats and physical noticeboards make it difficult to search reports, verify ownership, protect reporters' personal details, and keep a reliable history of resolutions. The project therefore provides a central, searchable and role-controlled workflow for lost-item reports, found-item reports, ownership claims, administrator review, and claim-status notifications.

The aim is to deliver a small but deployable and secure lost-and-found system that demonstrates disciplined software engineering practice within a 48-hour examination period. The objectives are to allow students to submit and browse reports; to enable evidence-based ownership claims; to give administrators control over claims and item status; to protect contact details until a claim is approved; and to provide students with visible feedback about their own reports, claims, and notifications.

1.2 2. Stakeholders and Scope

Stakeholder	Interest and responsibilities
Students	Report lost or found property, browse public listings, submit ownership claims, track their activity, and read claim decisions.
Administrators	Review reports and claims, approve or reject claims, manage the exact item statuses, and ensure unsuitable records are archived.
University/student-services office	Benefits from a structured, auditable alternative to informal lost-property handling.
Examiner	Evaluates the complete lifecycle evidence, working deployment, source code, and the student's understanding.

The 48-hour **minimum viable product** comprises secure sign-in, student/admin access controls, lost and found reporting with optional image storage, public item discovery, claim submission, admin review, in-app notifications, profile history, and responsive feedback states. Deferred work is recorded as explicit technical debt rather than being represented as completed functionality.

1.3 3. Software Requirements Specification

1.3.1 3.1 Functional Requirements

ID	Priority	Requirement	Acceptance criterion
FR-01	Must	The system shall authenticate users and distinguish the student and admin roles.	An unauthenticated user cannot access protected actions; an admin-only action returns an authorization error for a student.
FR-02	Must	A student shall create a lost-item report with title, description, category, date, location, and optional image.	A valid submission creates an item whose status is exactly lost; missing required fields produce validation feedback.
FR-03	Must	A student shall create a found-item report with the same data fields.	A valid submission creates an item whose status is exactly found.
FR-04	Must	Public users shall browse item listings with keyword, category, exact-status, and date filtering plus pagination.	The listing shows only matching items and reports a clear empty state when no result exists.
FR-05	Must	An authenticated student shall submit one ownership claim per eligible found item with proof-of-ownership text.	Invalid, duplicate, self, and non-found-item claims are refused; a valid claim is stored as pending.
FR-06	Must	An administrator shall review a claim and approve or reject it.	The decision is persisted, shown on the claimant profile,

ID	Priority	Requirement	Acceptance criterion
			and creates an in-app notification.
FR-07	Must	An administrator shall update an item status only to lost, found, resolved, or archived.	Any other status is rejected by server-side validation.
FR-08	Must	The system shall conceal an item reporter's contact details until the viewer has an approved claim for that item or is an administrator.	An unapproved public/student view contains no contact information; the approved claimant sees it.
FR-09	Must	A signed-in user shall view their submitted reports, ownership claims, and notification history.	The profile contains the user's records and distinguishes read from unread notifications.
FR-10	Should	The user interface shall display contextual success/error toast notifications after every mutation.	Reporting, claiming, decision, status update, image-upload failure, and notification-read actions emit a toast.

1.3.2 3.2 Non-Functional Requirements

ID	Requirement	Verification method
NFR-01	The interface shall be responsive for mobile, tablet, and desktop screen widths.	Manual viewport testing and responsive screenshot review.
NFR-02	All protected decisions shall be enforced by the server, not merely hidden in the client.	Procedure-level authorization tests.

ID	Requirement	Verification method
NFR-03	Text inputs shall be validated on both client and server; uploaded images shall have an approved image MIME type and a bounded size.	Unit/integration tests and manual negative testing.
NFR-04	Reporter contact information shall be disclosed only following the approved-claim policy.	Detail-query authorization tests.
NFR-05	Search results shall be paginated and the system shall present meaningful loading, empty, and error states.	Functional tests and manual browser verification.
NFR-06	Core user actions shall be understandable with keyboard-accessible controls, visible focus styles, labelled form fields, and sufficient text contrast.	Accessibility-oriented manual review.

1.4 4. Prioritisation and Effort Estimation

The project applies **expert estimation with a work-breakdown structure**, selected because the individual project has a fixed 48-hour time box, small scope, and no historical organisational data needed for calibrated Function Point or COCOMO estimates. Each deliverable is estimated in person-hours, then bounded by the examination schedule. The estimate consciously reserves time for testing, deployment verification, technical-debt analysis, and documentation rather than allocating all available time to coding.

Work package	Estimate (hours)	Assumption and scope implication
Requirements, SRS, backlog, estimation	4.0	A concise domain with two user roles is sufficient.
Analysis and design artefacts	3.5	Only decision-useful diagrams are produced.
Data model, authentication, authorization	5.0	Existing managed OAuth and database template are used.
Item reporting, optional image upload, validation	6.0	One image per report; no image editing or computer vision.

Work package	Estimate (hours)	Assumption and scope implication
Listing, filtering, pagination, item details	5.0	Fixed page size and indexed/limited queries.
Claims, admin management, notifications, profile	8.5	Notifications are in-app rather than email/SMS/push.
Responsive interface, feedback states, polish	4.5	Reusable UI components minimise duplicated work.
Automated and manual testing, defect correction	4.0	Tests focus on critical rules and workflows.
Deployment check, user manual, technical-debt and maintenance documentation	5.5	Deployment uses the project platform and documented test credentials.
Total	46.0	Leaves 2.0 hours contingency for integration defects, build issues, and final verification.

The estimate makes several constraints explicit: the project does not include real-time chat, campus-location mapping, public account registration, e-mail delivery, multiple image galleries, or automated ownership verification. Those features are potential future evolution, not hidden incomplete work.

1.5 5. High-Level Architecture and Core Data Model

FindBack uses a layered, full-stack web architecture: a React client provides public browsing and authenticated dashboards; typed server procedures validate all requests and enforce roles; a relational database holds user, item, claim, and notification metadata; and managed object storage retains optional image files. No reporter contact details are returned from the detail query unless the caller is an administrator or has an approved claim for that specific item.

Figure 1. FindBack system architecture. The rendered architecture diagram is supplied as `diagrams/system_architecture.png` in the supporting files.

Figure 1. The React client communicates with server-side tRPC procedures, which enforce authorization and coordinate the database, managed object storage and in-app notification records.

Figure 2. FindBack entity-relationship model. The rendered entity-relationship diagram is supplied as `diagrams/entity_relationship.png` in the supporting files.

Figure 2. Core data entities and relationships for user accounts, reports, ownership claims and notifications.

1.6 6. Security and Privacy Decisions

Authentication state is managed by the platform session rather than hand-written password storage. Role checks occur in server procedures. Server-side schemas validate all business inputs, limit date/filter values, and restrict uploaded content to safe image MIME types and bounded file sizes. The contact-disclosure policy is enforced at query composition so it cannot be bypassed by modifying the client. Public item listings return only the information necessary for discovery and never return private reporter contact data.

1.7 7. Initial Technical-Debt Register

Debt	Cause	Impact	Priority	Proposed resolution
In-app only decision alerts	48-hour scope constraint	Users must visit FindBack to read a decision.	Medium; scheduled	Add verified e-mail/push delivery with notification preferences in version 1.1.
Single-image report attachment	Simpler storage and validation workflow	Multiple photos cannot support complex property reports.	Low; acceptable temporarily	Introduce an <code>item_images</code> relation and gallery upload in version 1.1.
No automated image/content moderation	No moderation service budget in the exam scope	An admin must handle unsuitable reports manually.	Medium; scheduled	Add content moderation, reporting, and audit workflow before broad public rollout.
Simplified search	Small project data volume	Search does not include typo tolerance or ranking.	Low; acceptable temporarily	Add full-text search and database indexes after usage data justifies them.
Manual user-acceptance test cohort	Individual examination time limit	Feedback is limited to a small representative test.	Medium; scheduled	Run structured student-services pilot with feedback collection.

1.8 8. Implementation Summary

FindBack was implemented as a full-stack React, Express, tRPC, Drizzle, relational-database and managed-object-storage application. Platform OAuth provides authenticated sessions; server procedures apply role checks, validate all business input, and return only authorised data. Student workflows include public discovery, report creation, image attachment, claim submission, profile history and notifications. The administrator workspace reviews claims, writes optional decision notes, updates item status, and triggers in-app notifications.

The four permitted item statuses are enforced in the schema and server validation: lost, found, resolved, and archived. Claim status is separate and constrained to pending, approved, and rejected. Reporter contact data is removed from an item-detail response unless the requester is an administrator or has an approved claim on that exact item.

1.9 9. Testing and Quality Assurance

The automated test suite contains six passing tests across three files. It verifies the exact allowed item statuses, the privacy rule for reporter contact disclosure, claim eligibility, logout behaviour, anonymous report rejection, and student rejection from an administrator-only status update. A production build completed successfully. Responsive system inspection was performed at desktop and 375 × 812 mobile viewport sizes for the home, browse, report, profile and administrator pages.

The detailed test cases, results, defects, and manual live-testing checklist are supplied in **Testing_Report.md**. The manual end-to-end section must be completed on the published URL with separate student and administrator accounts immediately before submission; it must not be marked as passed without execution.

1.10 10. User Manual

Public users browse and filter reports. Signed-in students choose **Report lost** or **Report found**, provide the required information, optionally attach a permitted image, and publish. A student who recognises an active found item provides private proof of ownership. An administrator reviews that evidence, accepts or rejects the request, and FindBack creates an in-app notification for the claimant. The approved claimant may then view the reporter's contact information on the relevant item page.

Full step-by-step instructions are supplied in **User_Manual.md**.

1.11 11. Deployment and Accessibility

The project is configured as a deployable full-stack application. Before final submission, the student must create a final project checkpoint, use the project interface's **Publish** control, test the live application and / admin route, then paste the verified URL into **Deployment_and_Source_Links.txt**. The project must remain accessible for grading. The source repository URL should be exported and recorded in the same file. The interface uses responsive layouts, labelled inputs, keyboard-reachable controls, loading states, clear empty states, error recovery actions, visible focus treatment, and toast feedback. Search results are paginated. Sensitive reporter contact information is not included in public listings or unapproved item-detail results.

1.12 12. Technical Debt

The main technical debt is documented in **Technical_Debt_Plan.md**. The immediate priority before submission is not code refactoring; it is completion of the live manual test evidence and URL/credential verification. Subsequent releases should add delivery channels for notifications, referential constraints, multi-image support, moderation and improved search.

1.13 13. Maintenance Strategy

Corrective maintenance will address defects reported by students or administrators, using issue reproduction, automated regression tests and a small corrective release. Adaptive maintenance will track

browser, framework, authentication-platform and database changes. Perfective maintenance will prioritise improvements that remove friction identified through feedback, such as better filtering or notification preferences. Preventive maintenance includes dependency updates, vulnerability review, backup/retention policy confirmation, database-index review and regular access-control tests.

Each release should include a regression pass covering sign-in, role checks, report creation, image validation, browse filters, privacy disclosure, claim decisions, notifications and admin status controls. Metrics worth monitoring include report volume, claim approval time, unclaimed item ageing, failed image uploads, and user-reported recovery success.

1.14 14. Future Evolution

Future versions may introduce verified e-mail/push notifications, multiple photos, full-text ranked search, lost-property handover appointments, a campus map, content-reporting/moderation workflow, analytics, multilingual support, accessibility audit, richer administrator audit logs, and a structured data-retention policy. Features should be selected only after observing user and administrator needs, and all new work should enter an estimated and prioritised backlog.

1.15 15. Limitations

FindBack is intentionally a time-boxed capstone product rather than a large commercial platform. It does not send e-mail/SMS/push messages, conduct automated image moderation, support multiple images, or provide typo-tolerant search. These limitations are recorded transparently in the technical-debt plan. Final live credentials, published URL, source repository URL, and live acceptance-test results require the student's own deployment actions and must be verified before Sakai submission.

1.16 16. Conclusion

FindBack demonstrates a complete, scoped advanced software engineering lifecycle: requirements and prioritisation, effort estimation, architecture and data modelling, implementation, security and privacy controls, tests, technical-debt management, deployment preparation, user documentation, maintenance planning and future evolution. Its most important quality decision is the server-enforced policy that protects reporter contact details until a legitimate ownership claim is approved.

1.17 17. References

The application uses React, Express, tRPC, Drizzle ORM, Tailwind CSS, Zod and managed platform authentication/storage. Record the exact framework/library versions from package.json and any additional resources used during implementation in the final submitted reference list. No external datasets were used.